

115 學年度資訊學科能力競賽臺南一中 校內初選

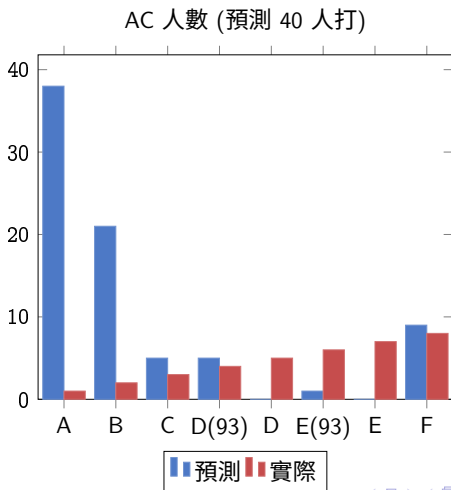
題解

Sep. 10 2026

預測

預計解出來的人數

預測複賽線: 260 分，實際複賽線 (rk10): TODO 分



難度預測

1 yushiuan9499: $A < B < C \approx F < D < E$

2 tobiichi3227:

$$A < B < C \approx F < D(93) < E(93) < D(100) < E(100)$$

tobiichi 的個人分數預測

- 1 李柏霆: 579
- 2 柯伯亨: 538
- 3 謝明吾: 538
- 4 黃鼎佑: 493

Fun Fact

Fun Fact – pA sample

pA 的範測二跟範測三分別是 2026 TOI 一模跟二模成績

然後二模第 20 小剛好是 67，笑死

Fun Fact – pC

- 12 個子任務，超級多

Fun Fact – pD

- 模數給 1145141 是因為 114514 然後還以為會用到質數的性質。
- OEIS A027423

- 分蛋糕系列第四題

Fun Fact – Appendix

因為不知道要放什麼在附錄

所以拿會考數學參考公式放在上面。

A. 大家好我是今年選訓二階結訓選手

author : blame & tobiichi3227

首殺: TODO

A. 大家好我是今年選訓二階結訓選手 – 題目敘述

給你一個長度為 n 的陣列 A

找其中由小到大排序，第 k 小的值是多少。

A. 大家好我是今年選訓二階結訓選手 – 子任務

1 $0 \leq A_i \leq 10^9$

2 無額外限制

A. 大家好我是今年選訓二階結訓選手 – $0 \leq A_i \leq 10^9$

給忘記開 long long 的人 67 分

因為忘記開真的很 67

A. 大家好我是今年選訓二階結訓選手 – 無額外限制

這題有很多做法

- 1 $O(n^2)$: 跑 k 次找最小值，每次找到最小值就在陣列上紀錄下來，然後下次就不要算到他。
- 2 $O(n^2)$: 跑 bubble sort，然後找第 k 小。
- 3 $O(n \log n)$: 跑 merge sort 或者 quick sort，找第 k 小。
- 4 $O(n)$: 用 `nth_element` 找第 k 小可以做到線性。

B. 好多手錶

author : tobiichi3227

首殺: TODO

B. 好多手錶 – 題目敘述

給一個 W 的陣列，值最多加到 43200，超過會 MOD 43200

求最少要增加多少次才會使整個陣列相同

B. 好多手錶 – 子任務

- 1 所有 t_i 相同
- 2 $W = 2$
- 3 $W \leq 8$
- 4 $W \leq 2000$
- 5 $W \leq 20000$
- 6 無額外限制

B. 好多手錶 – 所有 t_i 相同

送分的，拿不到可以考慮回家休息了

時間都一樣就不用調整了

B. 好多手錶 – $W = 2$

還是送分的，沒拿到可以檢討自己了

算兩個時間差，看是直接讓小的調整到大的比較好還是繞一圈比較好

B. 好多手錶 – $W \leq 8$

應該隨便做都有？

B. 好多手錶 – $W \leq 2000$

直接枚舉時間從 0 到 43199，並枚舉所有手錶

計算哪個時間調整過去所需花費最小

複雜度 $O(43200N)$

B. 好多手錶 – $W \leq 20000$

思考一下可以發現要調整到的時間會是某一支手錶原本的時間
為甚麼呢

考慮相鄰且不同的時間 $t_i < t_{i+1}$

選一個目標時間 x 使得 $t_i < x < t_{i+1}$ ，當 x 增加一秒時，所有 t_w 都要加一秒，整體增加 N

相鄰的區間內成本嚴格遞增，所以挑左端點是最好的，而左端點 t_i 必然存在於原本的時間中

B. 好多手錶 – $W \leq 5000$

基於前面所說的，我們可以直接枚舉所有手錶，每次選一個當作目標時間

計算哪個時間調整過去的花費最小

複雜度 $O(N^2)$

B. 好多手錶 – 無額外限制

這個有兩個做法，先講 $O(N \log_2 N)$ 的

先排序所有時間，挑一個目標時間 $x = t_i$ ， $t_1 \cdots t_i$ 調整到 t_i 不需要跨過 43200， $t_{i+1} \cdots t_W$ 需要跨過 43200

對於所有 $j \leq i$ 成本為

$$\sum_{j=0}^i (t_j - t_i) = (i+1)t_i - \sum_{j=0}^i a_j$$

對於所有 $j > i$ 成本為

$$\sum_{j=i+1}^{N-1} (43200 - (t_j - t_i)) = (N-i-1)(43200 + t_i) - \sum_{j=i+1}^{N-1} a_j$$

B. 好多手錶 – 無額外限制

可以發現後面那個 $\sum$ 就是前綴和

所以我們對原本的陣列求前綴和並枚舉所有時間就好了

複雜度 $O(N \log_2 N)$

B. 好多手錶 – 無額外限制

另一個更快的做法

令 $M = 43200$

定義

$$F(x) = \sum_{i=1}^N (x - t_i + M) \bmod M$$

，代表往前調到時間 x 的總成本

要找 $\min_{0 \leq x \leq M} \{F(x)\}$

假設我們知道 $F(x)$ ，從 x 增加到 $x + 1 \pmod{M}$ 一般來說總成本需要增加 N

但如果某個手錶已經在 $x + 1$ 了，那成本不同

B. 好多手錶 – 無額外限制

所以成本變化不是 $+1$ ，而是 $-(M - 1)$ ，總成本需要額外減去 M

如果時間為 $x + 1$ 的手錶有 $cnt[x + 1]$ 支，則
$$F(x + 1) = F(x) + N - M \cdot cnt[x + 1]$$

先算 $F(0)$

$$F(0) = \sum_{i=1}^N \begin{cases} 0, & \text{if } t_i = 0, \\ M - t_i, & \text{if } t_i > 0. \end{cases}$$

剩下的 $F(1), F(2), \dots, F(M - 1)$ 遞推算一下就有了

然後算的時候順便找最小值，複雜度 $O(N)$

B. 好多手錶 – 無額外限制

但這題沒打算卡到 $O(N)$ ，所以 $N \log_2 N$ 就可以過了

雖然 $O(N)$ 的程式應該更好寫

C. 撞球

author : blame

首殺: TODO

C. 撞球 – 題目敘述

給一張 $N \times M$ 的地圖 (會有外框)，每格會是空格或是 ‘/’ 或 ‘\’

給 Q 筆詢問，問從 a 的外框射進去最後會到哪一個外框

C. 撞球 – 子任務

- 1 $NM = 1$
- 2 $1 \leq Q \leq 10$
- 3 $N = 1$
- 4 $N = 2$
- 5 球檯內部只會有 ‘ 與 ‘/’ 或 ‘\’ 其中之一
- 6 所有詢問都從上方進入
- 7 每一列最多有一個 ‘\’ 或 ‘/’
- 8 $1 \leq N, M \leq 10, 1 \leq Q \leq 10$
- 9 $1 \leq N, M \leq 1000, 1 \leq Q \leq 100$ ，輸入的字元僅包含 ‘
- 10 $1 \leq N, M \leq 200, 1 \leq Q \leq 1000$
- 11 無額外限制

C. 撞球 – 前言

子任務太多

所以下面挑幾個子任務講

C. 撞球 – $N = 1$

發現地圖長成一行

所以每個路線最多只會遇到兩次鏡子

所以可以根據這些特性做出某些神奇的作法

C. 撞球 – $1 \leq N, M \leq 200, 1 \leq Q \leq 1000$

直接暴力模擬

複雜度 $O(Q \times (N + M))$

C. 撞球 – 無額外限制

觀察到:

性質

1. 沒有 -1 的情況。
2. 發現對於每個格子來說，最多只會被兩條路徑走到
3. 每條路徑的入口跟出口互為對方的答案

所以你發現可以轉成每個格子為四方位的一個四層圖，在上面好好 BFS 會過 (不好好做會被卡常)。

官解沒有像 BFS 一樣開 $O(4NM)$ 的空間，而是運用到第二條性質，所以你每次暴力模擬，然後遇到走過的入口或出口就紀錄答案下來，再次被詢問到就不跑暴力，直接回答。

D. 1145141

author : blame

首殺: TODO

D. 1145141 – 題目敘述

給定一個正整數 n ，求 $n!$ 的正因數個數。

$$1 \leq n \leq 10^9$$

D. 1145141 – 子任務

1 $1 \leq n \leq 5$

2 $1 \leq n \leq 15$

3 $1 \leq n \leq 10^3$

4 $1 \leq n \leq 10^6$

5 $1 \leq n \leq 10^8$

6 $1 \leq n \leq 10^9$

$$D. 1145141 - 1 \leq n \leq 5 \ \& \ 1 \leq n \leq 15$$

$$n \leq 5 :$$

$$ans = 1, 2, 4, 8, 16$$

$$D. 1145141 - 1 \leq n \leq 5 \ \& \ 1 \leq n \leq 15$$

$$n \leq 5 :$$

$$ans = 1, 2, 4, 8, 16$$

$$n \leq 15 :$$

$$15! \approx 10^{12}$$

所以可以先把 $n!$ 算出來之後再用 $O(\sqrt{n})$ 計算因數個數

$$D. 1145141 - 1 \leq n \leq 10^3$$

這邊有一個很關鍵的數學公式。

推論

當我們把 a 進行質因數拆解為

$$a = p_1^{k_1} \times p_2^{k_2} \times \dots \times p_n^{k_n}$$

因為對於每個質因數，可以選 $0 \sim k$ 個，
且每種選擇都可以對應到不同的因數。

所以對於 a 的正因數個數總和為

$$(k_1 + 1) \times (k_2 + 1) \times \dots \times (k_n + 1)$$

$$D. 1145141 - 1 \leq n \leq 10^3$$

那麼題目就轉化為了，對於一個質數 p 來講

他在 $n!$ 中總共貢獻多少次質因數？

$\Rightarrow n!$ 最多除多少次 p 才可以不被 p 整除

最後把 $[1, n]$ 中的質數貢獻 $+1$ 後全部乘起來就是答案了

更數學一點來講

$$ans = \prod_{p=1, p \text{ is prime}}^n (k+1), k \text{ is the maximum number such that } p^k | n!$$

$$D. 1145141 - 1 \leq n \leq 10^3$$

所以枚舉每個數字，用 $O(\sqrt{n})$ 判斷他是否為質數

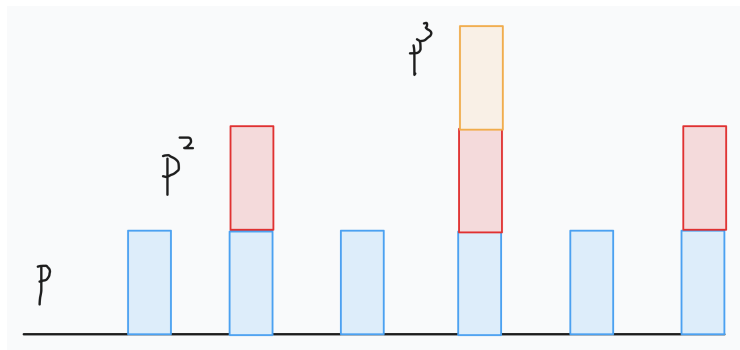
假如是質數，則跑過 $1 \sim n$ ，並計算總共除掉多少次。

對於每個數字最多會被 p 除掉 $\log(n)$ 次

花費時間 $O(n(\sqrt{n} + n \log(n))) = O(n^2 \log(n))$

$$D. 1145141 - 1 \leq n \leq 10^6$$

想像對於某個質數 p ，在數線上 k 的長相：



$$D. 1145141 - 1 \leq n \leq 10^6$$

發現只要 $k = 2$ 的地方就會被 $k = 1$ 的格子佔到欸?

雖然聽起來很像廢話，但是會發現我們可以將 $k = ?$ 分開算

$k = a$ 在 $n!$ 中貢獻很好算，就是 $\lfloor \frac{n}{a} \rfloor$

然後發現 k 最大也才 $\log(n)$ 量級

$$D. 1145141 - 1 \leq n \leq 10^6$$

發現只要 $k = 2$ 的地方就會被 $k = 1$ 的格子佔到欸?

雖然聽起來很像廢話，但是會發現我們可以將 $k = ?$ 分開算

$k = a$ 在 $n!$ 中貢獻很好算，就是 $\lfloor \frac{n}{a} \rfloor$

然後發現 k 最大也才 $\log(n)$ 量級

跑質數篩找質數 p ，用 $O(\log n)$ 的時間枚舉 k ，時間複雜度總共 $O(n + |\{p\}| \log n)$

$$D. 1145141 - 1 \leq n \leq 10^8 \ \& \ 1 \leq n \leq 10^9$$

剛剛的解法，空間開 $O(n)$ 掛了。

所以我們想要讓空間少一點。

$$D. 1145141 - 1 \leq n \leq 10^8 \ \& \ 1 \leq n \leq 10^9$$

剛剛的解法，空間開 $O(n)$ 掛了。

所以我們想要讓空間少一點。

首先，先分兩個 case 討論：

- $p \leq \sqrt{n}$
- $p > \sqrt{n}$

$$D. 1145141 - 1 \leq n \leq 10^8 \ \& \ 1 \leq n \leq 10^9$$

$$p \leq \sqrt{n}$$

因為我們可以花 $\sqrt{n}$ 的時間跟空間求出這些符合條件的質數 p 們

所以照剛剛解法求就好了。

$$D. 1145141 - 1 \leq n \leq 10^8 \text{ \& } 1 \leq n \leq 10^9$$

$$p \leq \sqrt{n}$$

因為我們可以花 $\sqrt{n}$ 的時間跟空間求出這些符合條件的質數 p 們

所以照剛剛解法求就好了。

$$p > \sqrt{n}$$

對於這些 p 來講，他們 $p^2 > n$ 欸!

那這樣不就是只要算 $(\sqrt{n}, n]$ 中的質數們 $\lfloor \frac{n}{p} \rfloor$ 的總和嗎?

$$D. 1145141 - 1 \leq n \leq 10^8 \text{ \& } 1 \leq n \leq 10^9$$

觀察到 $\lfloor \frac{n}{p} \rfloor$ 只有 $\sqrt{n}$ 個而已。

所以可以用數論分塊找到對於 $k = \lfloor \frac{n}{p} \rfloor$ 所對應的區間 $[l, r]$

也就是對於 $[l, r]$ 區間中的質數 p 們，他們的 k 都是 $\lfloor \frac{n}{l} \rfloor$

D. $1145141 - 1 \leq n \leq 10^8$ & $1 \leq n \leq 10^9$

觀察到 $\lfloor \frac{n}{p} \rfloor$ 只有 $\sqrt{n}$ 個而已。

所以可以用數論分塊找到對於 $k = \lfloor \frac{n}{p} \rfloor$ 所對應的區間 $[l, r]$

也就是對於 $[l, r]$ 區間中的質數 p 們，他們的 k 都是 $\lfloor \frac{n}{l} \rfloor$

數學中， $\pi(n)$ 為 $< n$ 總共有多少個質數。

這樣不就轉成了，數論分塊中的 $[l, r]$ ，對於答案的貢獻為
 $(k + 1)^{(\pi(r) - \pi(l-1))}$

$$D. 1145141 - 1 \leq n \leq 10^8 \text{ \& } 1 \leq n \leq 10^9$$

觀察到 $\lfloor \frac{n}{p} \rfloor$ 只有 $\sqrt{n}$ 個而已。

所以可以用數論分塊找到對於 $k = \lfloor \frac{n}{p} \rfloor$ 所對應的區間 $[l, r]$

也就是對於 $[l, r]$ 區間中的質數 p 們，他們的 k 都是 $\lfloor \frac{n}{l} \rfloor$

數學中， $\pi(n)$ 為 $< n$ 總共有多少個質數。

這樣不就轉成了，數論分塊中的 $[l, r]$ ，對於答案的貢獻為 $(k + 1)^{(\pi(r) - \pi(l-1))}$

那要怎麼求 $\pi(n)$?

D. $1145141 - 1 \leq n \leq 10^8$ & $1 \leq n \leq 10^9$

- 令 p_k 為從小到大數第 k 個質數
- $\text{lpf}(x)$ 是 x 最小的質因數

D. $1145141 - 1 \leq n \leq 10^8$ & $1 \leq k \leq 10^9$

- 令 p_k 為從小到大數第 k 個質數
- $\text{lpf}(x)$ 是 x 最小的質因數

然後通靈出一個 dp 函式。

$$dp(n, k) = \sum_{i=2}^n 1[\text{i is a prime or } \text{lpf}(i) > p_k]$$

這樣對於 $[2, n]$ 的質數個數 $\pi(n)$ ，就直接求 $dp(n, \infty)$ 就好了。

D. $1145141 - 1 \leq n \leq 10^8$ & $1 \leq n \leq 10^9$

不難發現到，其實只要求到 $dp(n, \leq \sqrt{n}$ 最大的質數的 rank)

而對於 $dp(n, 0)$ 的初始狀態，肯定是 $[2, n]$ 的所有數了，所以是 $dp(n, 0) = 2$

而這個 dp 其實就是慢慢從第一、第二個 ... 質數篩上去，所以對於 $dp(n, j)$ 的轉移，其實可以看成

$$dp(n, j) = dp(n, j - 1) - (dp(\lfloor \frac{n}{p_j} \rfloor, j - 1) - (j - 1))$$

從 $dp(n, j - 1)$ 轉上來，要把那些 $lpf(x) = p_j$ 的篩掉，阿那些數量是 $dp(\lfloor \frac{n}{p_j} \rfloor)$

但因為 dp 的定義，所以需要把 $1 \sim \lfloor \frac{n}{p_j} \rfloor$ 的質數刪掉，那些剛好等於 $j - 1$

$$D. 1145141 - 1 \leq n \leq 10^8 \ \& \ 1 \leq n \leq 10^9$$

最後用 bottom up 的形式

從 $j = 0$ 慢慢做背包上去算 dp

$$D. 1145141 - 1 \leq n \leq 10^8 \ \& \ 1 \leq n \leq 10^9$$

最後用 bottom up 的形式

從 $j = 0$ 慢慢做背包上去算 dp

恭喜你，發明了 min25 篩!

$$D. 1145141 - 1 \leq n \leq 10^8 \ \& \ 1 \leq n \leq 10^9$$

然後你開開心心的丟上去，你就會發現 TLE 在 $1 \leq n \leq 10^9$ 的 case 了！

因為你每次整數分塊 $O(\sqrt{n})$ 又搭配上 min25 篩的 $O(\frac{n^{\frac{3}{4}}}{\log(n)})$ ，變成了 $O(n^{\frac{7}{8}})$ ！

所以你要改成 top down，然後用 unordered_map 存。

或者跑一次 bottom up (因為神奇的，你在整數分塊中需要的 $\pi(n)$ 的 n 集合剛好等於 min25 中算過的 n 集合) 就可以 AC 了！

E. 分蛋糕 4

author : blame

最高分: TODO

E. 分蛋糕 4 – 題目敘述

題目太長了

懶得打，自己去看題目。

E. 分蛋糕 4 – 子任務

- 1 $2 \leq n \leq 50, 1 \leq Q \leq 1000$
- 2 $2 \leq n \leq 200$
- 3 $2 \leq n \leq 800$
- 4 無額外限制

E. 分蛋糕 $4 - 2 \leq n \leq 50, 1 \leq Q \leq 1000$

你可以把整個陣列原封不動傳給 decoder

然後讓 decoder 自己算 + 回答

E. 分蛋糕 $4 - 2 \leq n \leq 200$

區間 DP 做完後，傳給 decoder

複雜度 $O(n^3)$

E. 分蛋糕 $4 - 2 \leq n \leq 800$

發現這題的 *cost* 其實有性質

可以用 knuth 做到 $O(n^2)$ 算

然後樸實無華的傳給 decoder 回答

E. 分蛋糕 4 – 無額外限制

這邊講做到 $|S|$ 為 $2 \times C_2^n$ 的作法

先用 knuth 算出答案後

然後發現對於每一列的 k 其實具有遞增性

所以你只要用 '0' 作為斷點，計算每個斷點中有多少個 '1'

代表著 k 相對於之前那個增加多少

E. 分蛋糕 4 – 無額外限制

這邊講做到 $|S|$ 為 2850423 的作法

你發現一列一列傳沒有完整用到 knuth 的性質

所以你用一個斜線一個斜線的方式傳 (傳的過程可以用一個大整數代表著整列的狀態)

然後數學計算一下發現是好的。

E. 分蛋糕 4 – 無額外限制

這邊講做到 $|S|$ 為 2850423 的作法

你發現一列一列傳沒有完整用到 knuth 的性質

所以你用一個斜線一個斜線的方式傳 (傳的過程可以用一個大整數代表著整列的狀態)

然後數學計算一下發現是好的。

假如你還要做到更佳，可以去問 brinton

F. 種馬拉車

author : blame

首殺: TODO

F. 種馬拉車 – 題目敘述

有一個長度為 $2n$ 的陣列 A

上面有著 $0 \sim n - 1$ 各兩個

每個回文區間段會對答案貢獻區間最大值

F. 種馬拉車 – 前言

題目叫做馬拉車，然後放在 pF

其實根本在搞人的

這題的定位在水題，只是問你有沒有勇氣開這題而已 omob

F. 種馬拉車 – 子任務

- 1 $1 \leq n \leq 100$
- 2 $1 \leq n \leq 2000$
- 3 無額外限制

F. 種馬拉車 – $1 \leq n \leq 100$

$O(n^2)$ 枚舉每個區間

花費 $O(n)$ 檢查是否為回文，且 $O(n)$ 計算最大值

F. 種馬拉車 – $1 \leq n \leq 2000$

$O(n^2)$ 枚舉每個區間

用 rolling hash 之類的神奇演算法 $O(1)$ 檢查是否為回文

用線段樹、sparse-table 之類的神奇算法 $O(1)$ or $O(\log n)$ 計算最大值

F. 種馬拉車 – 無額外限制

官解當然不可能叫你做馬拉車

你發現你剛剛根本沒有用到題目的性質：

性質

$0 \sim n - 1$ 各出現兩次

而回文有性質

性質

對於區間 (長度 > 2) $[l, r]$ 是回文，區間 $[l + 1, r - 1]$ 也一定是回文

所以你可以仔細思考一下，發現這題最多也就 $O(3n)$ 個回文而已

F. 種馬拉車 – 無額外限制

作法就是由每個回文中心開始往左右擴

假如擴增遇到左右不相同則停止擴增

最大值也可以 $O(1)$ 維護

所以複雜度 $O(n)$

F. 種馬拉車 – 無額外限制

作法就是由每個回文中心開始往左右擴

假如擴增遇到左右不相同則停止擴增

最大值也可以 $O(1)$ 維護

所以複雜度 $O(n)$

假如你覺得這是 $O(n^2)$ 但寫完發現過了，那你很幸運